

Methodology: Pipeline Design and Formal Definitions

Methodology: Pipeline Design and Formal Definitions I

This section describes the five-stage computational meta-analysis pipeline. Each stage corresponds to a tested, independently executable script that reads upstream outputs and produces structured artifacts. The pipeline extends the systematic literature analysis approach of Knight et al. [knight2022fep]—which combined manual annotation with ontology-based automated analysis—by substituting manual coding with fully automated, LLM-driven assertion extraction and citation-weighted hypothesis scoring. All code, configuration files, and reproducibility instructions—including a Dockerized execution environment to guarantee dependency isolation—are publicly available in the project repository (https://github.com/ActiveInferenceInstitute/act_inf_metaanalysis).

Pipeline Overview I

The five-stage pipeline is summarized in `tab:pipeline_stages`.

Table 1: Five-stage computational meta-analysis pipeline. Each stage corresponds to an independently executable script that reads upstream outputs and produces structured artifacts. Cross-references link to detailed methodology sections.

Stage	Script	Primary Input	Primary Output
1	01_literature_search.py	API queries	corpus.jsonl
2	02_meta_analysis_pipeline.py	corpus.jsonl	Classification, temporal, TF-IDF, NMF, citation network JSON
3	03_build_knowledge_graph.py	corpus.jsonl	nanopublications.jsonl, nanopublications.trig, scores
4	04_generate_figures.py	All Stage 2–3 JSONs	16 publication-ready PNGs
5	05_inject_variables.py	All output JSONs	Rendered manuscript Markdown

Scripts act as thin orchestrators that import methods from tested library modules and handle file I/O. All computation resides in the `src/` packages; no analysis logic is embedded in scripts.

End-to-end pipeline execution completes in under one hour on commodity hardware (excluding LLM extraction, which depends on model size and inference backend); all stochastic components use fixed random seeds for deterministic reproduction.

Reproducible Build Infrastructure I

The five-stage analysis pipeline described above is embedded within `template/` [Friedman2026TemplateReproducibleGenerative, FriedmanTemplateSoftware], an open-source Infrastructure-as-Code system for computational research that turns a full research compendium—code, data, tests, manuscript, and provenance—into a single, version-controlled, deterministically buildable repository with an enforced, test-gated publication pipeline. `template/` applies the principle of Infrastructure as Code to the research lifecycle, making the manuscript, test suite, and provenance chain independently verifiable. The system operationalizes FAIR4RS principles [wilkinson2016fair] and supply-chain-style provenance for manuscripts, targeting structural causes of the reproducibility crisis: fragmented workflows across LaTeX, notebooks, and ad-hoc scripts, lack of end-to-end testing, and no binding between code, data, figures, and the final PDF.

Reproducible Build Infrastructure II

The system employs a Two-Layer Architecture: a globally shared *infrastructure layer* (12 subpackages, approximately 150 Python modules) provides generic services—logging, rendering, validation, steganographic watermarking, reporting, and LLM integration—while self-contained *project workspaces* (including the present meta-analysis) carry their own `manuscript/`, `scripts/`, `src/`, `tests/`, `data/`, and `output/` directories, discovered purely by filesystem convention. An eight-stage build pipeline enforces an ordered sequence from environment setup through test execution (at least 90% coverage for project code, at least 60% for shared infrastructure), analysis execution, PDF rendering (Pandoc to LaTeX to XeLaTeX with biber), output validation, LLM review, and executive reporting. A Zero-Mock testing policy requires all tests to exercise real filesystem operations, real subprocess calls, and real computation—no `unittest.mock` doubles—making test adequacy a publication gate rather than a best-effort guideline. Cryptographic provenance is embedded in every PDF via SHA-256 hash manifests, PDF metadata injection, and optional QR codes linking back to the repository. A Documentation Quality standard